

Fine-Mapping Benchmarking Framework

Simulation Documentation

This document provides complete mathematical and procedural details for the data simulation pipeline used in the fine-mapping benchmarking framework. It covers genotype simulation, functional annotation generation, causal variant selection, phenotype generation under both sparse and sparse + infinitesimal models, and computation of summary statistics.

Each section maps directly to the R functions in the repository. Parameter names in monospace correspond to function arguments.

Contents

1. Overview of the Simulation Pipeline
2. Stage 1: Genotype Simulation
3. Stage 2: Functional Annotation Simulation
4. Stage 3: Causal Variant Selection
5. Stage 4: Phenotype Generation
 - 5.1 Model 1: Sparse Only
 - 5.2 Model 2: Sparse + Infinitesimal Effects
6. Stage 5: Summary Statistics
7. Benchmarking Orchestration
8. Complete Parameter Reference

1. Overview of the Simulation Pipeline

The simulation pipeline consists of five stages, executed in order. Genotypes are simulated once and reused across all parameter combinations. Phenotypes are then generated repeatedly for each combination of simulation parameters.

Stage	Function	Description
1. Genotypes	<code>simulate_genotypes()</code>	Simulate genotype matrices from 1000 Genomes haplotypes using <code>sim1000G</code>
2. Annotations	<code>simulate_phenotypes()</code>	Optionally generate synthetic functional annotations (binary or continuous)
3. Causal selection	<code>simulate_phenotypes()</code>	Select S causal variants, uniformly or weighted by annotations
4. Phenotype	<code>simulate_phenotypes()</code>	Generate phenotype vector y under sparse or sparse + infinitesimal model
5. Summary stats	<code>simulate_phenotypes()</code>	Compute marginal z-scores, effect sizes, standard errors, and LD matrix

The orchestration wrapper `run_simulation()` automates the full pipeline: it calls `simulate_genotypes()` once, then iterates over a grid of parameter combinations (S , ϕ , and optionally p_{causal}), generating `n_iter` independent replicates for each combination.

2. Stage 1: Genotype Simulation

Genotype matrices are simulated using the sim1000G R package, which generates realistic genotype data by recombining haplotypes from the 1000 Genomes Project Phase 3 reference panel. This preserves realistic patterns of linkage disequilibrium (LD).

2.1 Procedure

For each of the `n_regions` independent genomic regions, the following steps are performed:

1. **Read VCF.** Load variant data from a VCF file. By default, the bundled sim1000G example VCF is used (a region on chromosome 4). Users may supply their own VCF files for different genomic regions.
2. **Filter variants.** Remove variants with minor allele frequency (MAF) below `min_maf` (default: 0.01) or above `max_maf` (default: no upper filter).
3. **Subset to target p.** If the VCF contains more variants than the requested `p`, a random subset of `p` variants is selected (sorted by position). This introduces variation between regions when the same VCF is reused.
4. **Simulate individuals.** Generate `n` unrelated diploid individuals by recombining haplotypes from the reference panel. The sim1000G functions `startSimulation()` and `generateUnrelatedIndividuals()` are used.
5. **Retrieve genotypes.** Extract the $n \times p$ genotype matrix with entries in $\{0, 1, 2\}$ representing the number of minor alleles.
6. **Remove monomorphic SNPs.** Any variant that is monomorphic in the simulated sample ($\text{MAF} = 0$) is removed. The actual number of SNPs `p` may therefore be slightly less than requested.
7. **Standardise (optional).** If `standardise = TRUE` (default), each column of the genotype matrix is centred to mean 0 and scaled to variance 1. Letting x_j denote column `j` of the raw genotype matrix, the standardised column is $(x_j - \bar{x}_j) / \text{sd}(x_j)$.

2.2 Output

For each region, the output is a list containing:

Field	Type	Description
<code>X</code>	matrix ($n \times p$)	Standardised genotype matrix (mean 0, variance 1 per column)
<code>X_raw</code>	matrix ($n \times p$)	Raw genotype matrix with 0/1/2 coding
<code>n</code>	integer	Number of individuals
<code>p</code>	integer	Actual number of SNPs after filtering
<code>maf</code>	numeric (<code>p</code>)	Minor allele frequencies in the simulated sample
<code>variant_ids</code>	character (<code>p</code>)	Variant identifiers from the VCF
<code>region_id</code>	integer	Index of this region

vcf_source	character	Path to the VCF file used
------------	-----------	---------------------------

2.3 Parameters

Parameter	Default	Description
n_regions	3	Number of independent genomic regions to simulate
n	500	Number of unrelated individuals
p	200	Target number of SNPs per region. Scalar (same for all regions) or vector of length n_regions. Capped at 500 when using the bundled VCF.
vcf_files	NULL	Paths to VCF files. NULL uses the bundled sim1000G example. A single path is reused for all regions. A vector must have length n_regions.
min_maf	0.01	Minimum minor allele frequency for variant filtering
max_maf	NA	Maximum minor allele frequency. NA means no upper filter.
standardise	TRUE	If TRUE, standardise genotype columns to mean 0, variance 1
seed	NULL	Random seed for reproducibility

2.4 Linkage Disequilibrium

Because genotypes are generated by recombining real 1000 Genomes haplotypes, the simulated data exhibits realistic LD patterns. The LD matrix R is computed as the Pearson correlation matrix of the standardised genotype matrix:

$$R = \text{cor}(X)$$

For standardised X (each column with mean 0 and variance 1), this is equivalent to $R = (1/(n-1)) X^T X$. The LD matrix is computed once per region and shared across all simulation scenarios.

3. Stage 2: Functional Annotation Simulation

Functional annotations are optional. When enabled, they influence the probability of each variant being selected as causal. The annotation matrix $A \in \mathbb{R}^{p \times m}$ has one row per variant and one column per annotation, where m is the number of annotations.

3.1 No Annotations

When `annotations = "none"` (default), no annotation matrix is generated and causal variants are selected uniformly at random.

3.2 Binary Annotations

When `annotations = "binary"`, a binary annotation matrix $A \in \{0,1\}^{p \times m}$ is generated. Each entry A_{jk} indicates whether variant j belongs to annotation category k .

For each annotation $k = 1, \dots, m$, the proportion of variants with $A_{jk} = 1$ is determined by the `annotation_proportions` parameter:

- **NULL** (default): For each annotation, a proportion is drawn independently from $\text{Uniform}(0.01, 0.30)$. This produces a mix of rare and common annotations, loosely inspired by PAINTOR's simulation design (coding exons at 2.2%, DHS at 30.7%).
- **Scalar**: The same proportion is used for all annotations.
- **Vector of length m** : Each annotation gets its own specified proportion.

Given the proportions $\pi_1, \dots, \pi_m$, each entry is drawn independently:

$$A_{jk} \sim \text{Bernoulli}(\pi_k), \quad j = 1, \dots, p, \quad k = 1, \dots, m$$

A variant can belong to zero, one, or multiple annotation categories simultaneously.

3.3 Continuous Annotations

When `annotations = "continuous"`, following the approach of Funmap (Li et al.), each annotation value is drawn independently from a standard normal distribution:

$$A_{jk} \sim N(0, 1), \quad j = 1, \dots, p, \quad k = 1, \dots, m$$

3.4 User-Supplied Annotations

The user may supply their own annotation matrix directly via the `annotations` parameter. The matrix must have exactly p rows (matching the number of SNPs in each region). This enables the use of real biological annotations (e.g., from the baseline-LF model or DeepSEA features).

3.5 Annotation Parameters

Parameter	Default	Description
<code>annotations</code>	"none"	Type of annotations: "none", "binary", "continuous", or a user-supplied matrix

n_annotations	3	Number of annotation categories (m). Used for binary and continuous.
annotation_proportions	NULL	Proportion of variants with value 1 per annotation (binary only). NULL draws from Uniform(0.01, 0.30). Scalar or vector of length m.

4. Stage 3: Causal Variant Selection

Exactly S causal variants are selected from the p available variants. The selection mechanism depends on whether functional annotations are present.

4.1 Without Annotations (Uniform Selection)

When `annotations = "none"`, the S causal indices are drawn uniformly at random without replacement from $\{1, \dots, p\}$:

$$C \sim \text{Uniform sample of size } S \text{ from } \{1, \dots, p\}$$

Each variant has equal probability S/p of being selected.

4.2 With Annotations (Weighted Selection)

When annotations are present, causal variant selection is weighted by the annotation-enrichment model. This follows the logistic enrichment framework used by PAINTOR, SparsePro, CARMA, and Funmap.

Step 1: Determine enrichment values. The fold-enrichment for each annotation is controlled by the enrichment parameter:

- **NULL** (default): For each annotation, an enrichment is drawn from $\text{Uniform}(2, 10)$.
- **Scalar**: The same enrichment is used for all annotations.
- **Vector of length m** : Each annotation gets its own enrichment.

Step 2: Compute variant weights. For each variant j , an unnormalised weight is computed using a multiplicative model:

$$w_j = \exp\left(\sum_{k=1}^m A_{jk} \cdot \log(e_k)\right)$$

where e_k is the fold-enrichment for annotation k . For binary annotations, a variant with $A_{jk} = 1$ has its weight multiplied by e_k relative to a variant with $A_{jk} = 0$. For continuous annotations, a one-unit increase in A_{jk} multiplies the weight by e_k .

For numerical stability, the computation is performed as: $\log w_j = \sum_k A_{jk} \cdot \log(e_k) - \max_j(\sum_k A_{jk} \cdot \log(e_k))$, followed by $w_j = \exp(\log w_j)$.

Step 3: Normalise and sample. Weights are normalised to sampling probabilities:

$$\pi_j = w_j / \sum_{j'=1}^p w_{j'}$$

Then S causal variants are drawn without replacement with probabilities $\pi_1, \dots, \pi_p$.

4.3 Enrichment Parameters

Parameter	Default	Description
-----------	---------	-------------

enrichment	NULL	Fold-enrichment per annotation. NULL draws from Uniform(2, 10). Scalar applies the same enrichment to all annotations. Vector of length m gives per-annotation enrichments. Values > 1 indicate enrichment; values in (0,1) indicate depletion.
------------	------	---

5. Stage 4: Phenotype Generation

Given the standardised genotype matrix $X \in \mathbb{R}^{n \times p}$, the set of causal indices C with $|C| = S$, and the target proportion of variance explained $\phi \in (0,1)$, the phenotype vector $y \in \mathbb{R}^n$ is generated under one of two models.

5.1 Model 1: Sparse Only

This is the standard generative model used by SuSiE (Wang et al. 2020), FINEMAP (Benner et al. 2016), FiniMOM (Karhunen et al.), CARMA (Yang et al.), and Funmap (Li et al.).

$$y = X\beta + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I_n)$$

where $\beta \in \mathbb{R}^p$ is a sparse vector with exactly S non-zero entries.

5.1.1 Effect Size Distribution

Normal distribution (`effect_distribution = "normal"`, default). For each causal variant $j \in C$:

$$\beta_j \sim N(0, \sigma_\beta^2)$$

where σ_β^2 is controlled by `effect_variance` (default: 0.36, i.e. $\text{sd} = 0.6$, following SuSiE). For all $j \notin C$, $\beta_j = 0$.

Equal variance partition (`effect_distribution = "equal"`). All causal variants are assigned equal effect sizes of 1 (with random signs if needed to avoid a degenerate genetic signal). The overall scale is then determined by the variance calibration step below.

5.1.2 Variance Calibration

The residual variance σ^2 is calibrated so that the genetic component explains exactly the target proportion ϕ of phenotypic variance. Following SuSiE (Wang et al. 2020, equation 3–4):

$$\phi = \text{Var}(X\beta) / (\text{Var}(X\beta) + \sigma^2)$$

Solving for σ^2 :

$$\sigma^2 = \text{Var}(X\beta) \cdot (1 - \phi) / \phi$$

where $\text{Var}(\cdot)$ denotes the sample variance of the vector $X\beta$. This ensures that the PVE is exactly ϕ by construction for each simulated dataset. The calibration uses the empirical variance of the genetic signal from the actual genotypes and drawn effect sizes, not an expected value.

5.1.3 Phenotype Simulation

Given the calibrated σ^2 , the phenotype is drawn as:

$$y_i = \sum_{j=1}^p X_{ij}\beta_j + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2), \quad i = 1, \dots, n$$

5.2 Model 2: Sparse + Infinitesimal Effects

This model augments the sparse component with an infinitesimal polygenic background, following BEATRICE (Ghosal et al.) and SuSiE-inf (Cui et al.). The key motivation, established by Cui et al., is

that the purely sparse assumption underlying standard fine-mapping methods is the primary driver of PIP miscalibration in real data.

$$y = g_{\text{sparse}} + g_{\text{inf}} + \varepsilon$$

where $g_{\text{sparse}} = X\beta$ is the sparse genetic signal (as in Model 1), g_{inf} is the infinitesimal polygenic component, and ε is residual noise.

5.2.1 Infinitesimal Component: Two Formulations

BEATRICE formulation (`inf_model = "beatrice"`, default). The infinitesimal effects arise only from noncausal variants. Let X_{NC} denote the submatrix of X corresponding to the $m-S$ noncausal variants. Then:

$$\alpha_{\text{NC}} \sim N(0, (1/(m-S)) \cdot I_{m-S}), \quad g_{\text{inf}} = X_{\text{NC}} \alpha_{\text{NC}}$$

This follows Ghosal et al., where the noncausal component creates a challenging scenario in which noncausal variants have inflated GWAS statistics due to both LD with causal variants and their own small direct effects.

SuSiE-inf formulation (`inf_model = "susie_inf"`). The infinitesimal effects come from all variants (including causal ones):

$$\alpha_j \sim N(0, 1/p) \text{ for all } j = 1, \dots, p, \quad g_{\text{inf}} = X\alpha$$

This is the model assumed by the SuSiE-inf inference procedure.

5.2.2 Variance Normalisation

Following BEATRICE (equation 21 in the benchmarking document), the total phenotypic variance is decomposed into three components, controlled by two parameters: ϕ (total PVE) and p_{causal} (proportion of genetic variance from the sparse component).

- Sparse component explains $p_{\text{causal}} \cdot \phi$ of total phenotypic variance
- Infinitesimal component explains $(1 - p_{\text{causal}}) \cdot \phi$ of total phenotypic variance
- Residual noise explains $1 - \phi$ of total phenotypic variance

The phenotype is constructed by scaling each component to achieve the target variance decomposition:

$$y^{\blacksquare} = \sqrt{(p_{\text{causal}} \cdot \phi / \text{Var}(g_{\text{sparse}}))} \cdot g_{\text{sparse}} + \sqrt{((1-p_{\text{causal}}) \cdot \phi / \text{Var}(g_{\text{inf}}))} \cdot g_{\text{inf}} + \varepsilon^{\blacksquare}$$

where $\varepsilon^{\blacksquare} \sim N(0, (1-\phi) \cdot I_n)$ and $\text{Var}(\cdot)$ denotes the sample variance. The true sparse effect sizes β are rescaled by the same factor applied to g_{sparse} , so that the ground truth remains consistent.

Note that because both g_{inf} and ε are random, the realised PVE will fluctuate around the target ϕ in any single replicate. With larger n , the realised PVE converges to the target.

5.3 Phenotype Parameters

Parameter	Default	Description
-----------	---------	-------------

s	c(1,2,3,5)	Number of causal variants. Scalar or vector (swept in run_simulation).
phi	c(0.1,0.2,0.4,0.6)	Total PVE. Scalar or vector (swept in run_simulation).
model	"sparse"	"sparse" for sparse-only model; "sparse_inf" for sparse + infinitesimal.
p_causal	c(0.1,0.2,0.4)	Proportion of genetic variance from sparse component. Only used when model = "sparse_inf". Swept in run_simulation.
inf_model	"beatrice"	Infinitesimal formulation: "beatrice" (noncausal variants only) or "susie_inf" (all variants).
effect_distribution	"normal"	"normal" draws from $N(0, \text{effect_variance})$; "equal" assigns equal effect sizes.
effect_variance	0.36	Variance of the normal effect size distribution (σ^2_β). Default corresponds to sd = 0.6 (SuSiE).

6. Stage 5: Summary Statistics

After generating the phenotype vector y , marginal summary statistics are computed for use by summary-statistic-based fine-mapping methods.

6.1 Marginal Effect Sizes and Standard Errors

For each variant $j = 1, \dots, p$, a marginal ordinary least squares (OLS) regression of y on x_j (with implicit intercept) is performed. Since X is standardised (mean 0), centring y is equivalent to including an intercept. The marginal effect estimate is:

$$\beta_{\blacksquare_j} = (x_j^T y_{\blacksquare}) / (x_j^T x_j)$$

where $y_{\blacksquare} = y - \text{mean}(y)$ is the centred phenotype. The residual variance and standard error are:

$$\sigma_{\blacksquare_j}^2 = \sum_i (y_{\blacksquare_i} - x_{ij} \beta_{\blacksquare_j})^2 / (n - 2)$$

$$\text{se}(\beta_{\blacksquare_j}) = \sqrt{(\sigma_{\blacksquare_j}^2 / (x_j^T x_j))}$$

The denominator $n-2$ accounts for two degrees of freedom (intercept and slope).

6.2 Z-Scores

The z-score for variant j is the Wald statistic:

$$z_j = \beta_{\blacksquare_j} / \text{se}(\beta_{\blacksquare_j})$$

Under the null hypothesis of no association, z_j follows approximately a standard normal distribution.

6.3 LD Matrix

The LD matrix $R \in \blacksquare^{p \times p}$ is the Pearson correlation matrix of the genotype matrix X , computed using $\text{cor}(X)$. This is computed once per region (from the genotypes) and is shared across all simulation scenarios, since it depends only on the genotype data, not the phenotype.

6.4 Output Fields

Field	Type	Description
<code>z</code>	numeric (p)	Marginal z-scores
<code>beta_hat</code>	numeric (p)	Marginal OLS effect size estimates
<code>se</code>	numeric (p)	Standard errors of marginal effects
<code>LD</code>	matrix (p × p)	LD (correlation) matrix, stored in genotypes (computed once)

7. Benchmarking Orchestration

The `run_simulation()` function orchestrates the full simulation pipeline for benchmarking. It generates genotypes once, then sweeps over a grid of parameter combinations to produce multiple phenotype replicates.

7.1 Parameter Grid

The function forms the Cartesian product of all parameter arrays:

Sparse model: The grid is $S \times \phi \times n_iter$. With defaults $S = \{1,2,3,5\}$, $\phi = \{0.1, 0.2, 0.4, 0.6\}$, and $n_iter = 5$, this produces $4 \times 4 \times 5 = 80$ scenarios per region.

Sparse + infinitesimal model: The grid is $S \times \phi \times p_{causal} \times n_iter$. With defaults adding $p_{causal} = \{0.1, 0.2, 0.4\}$, this produces $4 \times 4 \times 3 \times 5 = 240$ scenarios per region.

Each scenario is an independent replicate: causal variant positions, effect sizes, infinitesimal effects (if applicable), and residual noise are all redrawn.

7.2 Output Structure

The output is a list with three elements:

genotypes: The output of `simulate_genotypes()`, containing genotype matrices, MAFs, variant IDs, and the pre-computed LD matrix for each region. This is shared across all scenarios.

scenarios: A flat list of simulation scenarios. Each scenario contains the parameter values (S , ϕ , p_{causal} , iteration number) and, for each region, the phenotype-specific outputs (y , z , β , se , annotations, ground truth). Genotype data is not duplicated.

params: A record of all simulation parameters for reproducibility.

7.3 Orchestration Parameters

Parameter	Default	Description
<code>n_iter</code>	5	Number of independent replicates per parameter combination
<code>S</code>	<code>c(1,2,3,5)</code>	Vector of causal variant counts to sweep
<code>phi</code>	<code>c(0.1,0.2,0.4,0.6)</code>	Vector of PVE values to sweep
<code>p_causal</code>	<code>c(0.1,0.2,0.4)</code>	Vector of sparse variance proportions to sweep (sparse_inf only)

7.4 Reproducibility

A master seed controls the entire simulation. When set, the sequence of random draws is deterministic: genotype simulation uses the initial RNG state, and each subsequent phenotype scenario continues sequentially from the same RNG stream. This ensures full reproducibility: the same seed always produces the same genotypes, causal selections, effect sizes, and phenotypes.

8. Complete Parameter Reference

This section provides a consolidated reference of all user-facing parameters across the three main functions.

8.1 simulate_genotypes()

Parameter	Default	Description
n_regions	3	Number of independent genomic regions
n	500	Number of unrelated individuals
p	200	SNPs per region (scalar or vector). Max 500 with bundled VCF.
vcf_files	NULL	VCF file paths. NULL = bundled example.
min_maf	0.01	Minimum MAF filter
max_maf	NA	Maximum MAF filter (NA = none)
standardise	TRUE	Standardise genotype columns to mean 0, var 1
seed	NULL	Random seed
verbose	TRUE	Print progress messages

8.2 simulate_phenotypes()

Parameter	Default	Description
genotypes	(required)	Output from simulate_genotypes()
S	1	Number of causal variants (scalar or per-region vector)
phi	0.1	Total PVE (scalar or per-region vector), must be in (0,1)
model	"sparse"	"sparse" or "sparse_inf"
p_causal	0.5	Sparse variance proportion (sparse_inf only), in (0,1]
inf_model	"beatrice"	"beatrice" or "susie_inf"
effect_distribution	"normal"	"normal" or "equal"
effect_variance	0.36	Variance of normal effect sizes (sd=0.6)
annotations	"none"	"none", "binary", "continuous", or matrix
n_annotations	3	Number of annotation categories

annotation_proportions	NULL	Binary annotation proportions. NULL = random.
enrichment	NULL	Fold-enrichment per annotation. NULL = random.
seed	NULL	Random seed
verbose	TRUE	Print progress messages

8.3 run_simulation()

Accepts all parameters from both `simulate_genotypes()` and `simulate_phenotypes()`, plus the following orchestration parameters:

Parameter	Default	Description
n_iter	5	Replicates per parameter combination
S	c(1,2,3,5)	Causal variant counts to sweep
phi	c(0.1,0.2,0.4,0.6)	PVE values to sweep
p_causal	c(0.1,0.2,0.4)	Sparse variance proportions to sweep (sparse_inf only)

Note: In `run_simulation()`, the `S` and `phi` parameters are interpreted as arrays to sweep over (forming a grid), unlike in `simulate_phenotypes()` where they specify a single value (or per-region vector) for one simulation.